

NBA Game Visualizer

NBA_game_visualizer_spec.md: this document

General Description

This app lets a basketball fan look at how the score gap between the two teams rose and fell play by play. A user picks a game, and the app shows a line chart of the score differential over the full game so they can immediately see when things got tense, when a team pulled away, or where a comeback happened. The app pulls play-by-play data for a specific game and processes it into a momentum line. It also identifies the biggest single-play momentum swings and highlights them on the chart. For Version 1, this runs as a command line script. The user sets a game ID at the top of main.py, runs the script, gets a brief console summary, and a plotly chart opens in the browser.

- External:
 - ESPN's unofficial public API (site.api.espn.com) — no API key needed, free, confirmed working via instructor's example notebook
 - requests to call the API
 - pandas to process the play-by-play data into a dataframe
 - plotly to build the interactive momentum chart
- GUI:
 - For Version 1, CLI only (hardcoded game ID, chart opens in browser via fig.show()). Eventually I'd like a simple Streamlit web app where the user picks a team and date range and the chart updates interactively.
- Possible enhancements:
 - Let user pick a team and browse recent games instead of entering a raw game ID
 - Hover tooltips showing the exact play description at any point on the chart
 - Overlay two games' momentum curves to compare similar comebacks
 - Streamlit frontend for a cleaner UI

Task Vignettes

Task 1: Load and process a game

The user opens main.py, sets GAME_ID = "401873202" at the top, and runs the script. Within a few seconds the script prints to the console: both team names, the final score, and how many plays were found. In the background it builds a clean table of all the plays and saves it as a CSV so I don't have to re-call the API every time I test.

Calls ESPN summary endpoint:

http://site.api.espn.com/apis/site/v2/sports/basketball/nba/summary?event={GAME_ID}

- Team names/IDs pulled from header.competitions[0].competitors; play list from top-level plays key
- Each play flattened into a row: period, clock, team, type, text, scoreValue, homeScore, awayScore, scoringPlay
- List of dicts -> pandas dataframe, saved to data/play_by_play_{GAME_ID}.csv
- Caveat: unofficial API, undocumented, could change. Need .get() with defaults throughout so missing keys don't crash the script.

Task 2: View the momentum chart and biggest swings

After the data loads, the script adds a momentum column (home score minus away score), finds the 5 plays where that number changed the most in a single play, prints those to the console as a short list, then opens the chart. The line shows the whole game arc, positive = home team leading, negative = away team leading. The top swing plays are marked on the line.

- momentum = df["homeScore"] - df["awayScore"]
- Top swings: df["momentum"].diff(), sort by absolute value, take top 5
- Chart: plotly line (x = play order, y = momentum), top swings marked as scatter points with play text in hover
- fig.show() opens in browser; can also use fig.write_html("output.html") for saving

Technical Flow

Data structures:

- Raw JSON response (nested dict) from ESPN — play list and game header
- Dict mapping team ID -> team name (built from header)
- List of flat dicts (one per play) with readable fields
- Pandas dataframe built from that list, gains a momentum column and a swing column
- Plotly Figure object for the chart
- CSV file saved to data/ as a side effect

Core functions (rough sketch):

- `fetch_game_data(game_id)` — calls ESPN endpoint, returns raw JSON or prints error and exits
- `build_team_map(header)` — returns `{team_id: team_name}` dict from the header section
- `parse_plays(plays, team_map)` — loops over play list, returns list of flat dicts with team names filled in
- `compute_momentum(df)` — adds momentum and swing columns, returns updated df
- `find_top_swings(df, n=5)` — returns small df of the n biggest single-play swings
- `plot_momentum(df, top_swings)` — builds and returns a plotly Figure
- `save_csv(df, path)` — writes df to disk

Program flow:

- `main.py` runs top to bottom: `fetch` -> `parse` -> `dataframe` -> `momentum` -> `top swings` -> `print summary` -> `chart` -> `save CSV`
- No loops back to user during a run (v1 is a one-shot script, not interactive)

Final (Self) Assessment

- Biggest change from sketch: I was going to use Balldontlie or RapidAPI and build a real win-probability model. After the instructor's feedback and example notebook, I switched to ESPN's API for simplicity and replaced "win probability" with plain score differential, which I can actually compute from play-by-play data without a statistical model.
- I feel fairly confident about this spec because the instructor's notebook already shows the API call, the JSON structure, and the team-mapping step, so I'm mostly connecting pieces I can see working.
- Biggest potential problem: ESPN's JSON is deeply nested and undocumented. It might behave differently for overtime games or games where fields are missing. I'll need to test with several different game IDs before I trust the parser.
- Least familiar: plotly annotations and marking the top-swing plays on the chart without making it unreadable. May need help here.